Politechnika Warszawska
Wydział Geodezji i Kartografii
Geoinformatyka

Przedmiot: Algorytmy i struktury danych 2

Sprawozdanie

Analiza algorytmów na podstawie wyrażeń regularnych

Skład zespołu oraz nr indeksów:

Aleksandra Jabłońska 338739

Maksymilian Dźwigało 343533

Data oddania ćwiczenia: 3.11.2026

Semestr: 3

Rok akademicki: 2026/2027

1. Cel ćwiczenia

Celem ćwiczenia było napisanie programu w C++, który sprawdza, czy napis pasuje do wyrażenia regularnego. Rozwiązanie oparliśmy na automacie skończonym i rozwijaliśmy je w trzech krokach: od automatu deterministycznego (DFA), przez niedeterministyczny (NFA) zapisany tabelą, po NFA z przejściami epsilon zapisany jako graf.

2. Wstęp teoretyczny

Wyrażenia regularne to wzorce opisujące łańcuchy symboli, stosuje się je m.in. do wyszukiwania napisów w tekście i walidacji danych. Używamy konkatenacji, alternatywy **|** oraz domknięcia ***** (zero lub więcej powtórzeń).

Automat skończony ma skończoną liczbę stanów i reguły przejścia między nimi. W automacie **deterministycznym (DFA)** dla danego wejścia istnieje dokładnie jeden stan wyjściowy, więc automat zawsze jest w jednym, dobrze określonym stanie. W **niedeterministycznym (NFA)** z jednego stanu może prowadzić kilka przejść i automat może być w kilku stanach jednocześnie. Tekst pasuje do wzorca, jeśli uda się dojść do stanu końcowego.

NFA dla wyrażenia regularnego. Ogólny schemat: zbudować NFA odpowiadający wyrażeniu, a potem symulować jego działanie na tekście. NFA ma jeden stan na znak wzorca, a przejścia „epsilon” (bez czytania znaku) zapisuje się jako graf skierowany. Symulacja polega na śledzeniu zbioru stanów możliwych w danej chwili: po każdym znaku wykonujemy przejście po literze, a potem dodajemy stany osiągalne przejściami epsilon (osiągalność w grafie). Pesymistyczny czas rozpoznania tekstu o długości N dla wzorca o długości M jest proporcjonalny do N·M. Zbudowanie NFA zajmuje czas proporcjonalny do M.

3. Przebieg ćwiczenia

Ćwiczenie robiliśmy w trzech krokach i powstały z tego trzy programy.

Program 1 – automat deterministyczny (DFA). Wzorzec to A*(B|C)*B. Zrobiliśmy tablicę przejść, w której w każdej komórce jest dokładnie jeden stan. Mamy stany S0 (start), S1 (ostatnia litera to B, stan końcowy), S2 (ostatnia litera to C) i TRAP (napis nie pasuje). Litery spoza wzorca trafiają do kolumny „INNE” i od razu do stanu TRAP. Program czyta napis znak po znaku.

Program 2 – automat niedeterministyczny (NFA). Wzorzec A*(B|C)*AC. Po literze A nie wiadomo, czy to A*, czy końcowe „A” z „AC”, więc z jednego stanu trzeba pójść do dwóch stanów naraz. Dlatego w tabeli w każdej komórce jest lista stanów, a tablica pokazuje, w których stanach automat może teraz być. Dodaliśmy też wyjście z pętli, gdy żaden stan nie jest aktywny.

Program 3 – automat niedeterministyczny (NFA). Automat jest grafem - każdy stan ma listę krawędzi, a krawędź ma literę albo jest przejściem epsilon (przejście bez czytania litery). Dzięki czemu ***** i **|** zapisujemy krawędziami, a nie łączymy w jeden stan jak w programie 2. Dla każdego znaku przechodzimy po literze i dodajemy stany, do których da się dojść przejściami epsilon. Zbiór set pilnuje, żeby się nie zapętlić.

4. Wyniki i obserwacje

Program	Wzorzec	Napis	Wynik	Obserwacje
1	A*(B|C)*B	B	pasuje	Dowolna liczba liter A (też zero), dowolna liczba liter B lub C, cały napis musi się kończyć literą B.
2	A*(B|C)*AC	AAC	pasuje	Dowolna liczba liter A (też zero), dowolna liczba liter B lub C, cały napis musi się kończyć AC.
2	A*(B|C)*AC	CBAC	pasuje	Dowolna liczba liter A (też zero), dowolna liczba liter B lub C, cały napis musi się kończyć literą AC.
2	A*(B|C)*AC	“ “	nie pasuje	Pusty napis, napis musi się kończyć literą AC.
2	A*(B|C)*AC	AABC	nie pasuje	Napis kończy się na BC, a nie na AC.
3	A*(B|C)*AC	BAC	pasuje	Dowolna liczba liter A (też zero), dowolna liczba liter B lub C, cały napis musi się kończyć literą AC.
3	A*(B|C)*AC	BCAAC	nie pasuje	Po literach B i C może wystąpić jedynie “AC”.
3	A*(B|C)*AC	ACAC	pasuje	Dowolna liczba liter A (też zero), dowolna liczba liter B lub C, cały napis musi się kończyć literą AC.
3	A*(B|C)*AC	ADAC	nie pasuje	Litera D nie występuje w napisie.

5. Wnioski

Program 1 (DFA) jest wystarczający dla wzorca A*(B|C)*B, bo po każdej literze wiadomo, jaki jest następny stan. Dla wzorca A*(B|C)*AC już nie, bo po literze A są dwie możliwości.

Program 2 (NFA) zapamiętuje wszystkie stany, w których może być automat, dzięki czemu radzi sobie z bardziej rozwiniętym wzorcem. Wadą tego rozwiązania jest ułożenie tabeli, które działa poprawnie tylko dla jednego wzorca.

Program 3 (NFA) jest rozwiązaniem, które wymaga większego wysiłku. Zaletą algorytmu jest jego łatwiejsze zastosowanie w przypadku innych wyrażeń. Zamiast używania przejść opierających się o wcześniejszym zbadaniem wyrażenia, program rozkłada kroki na prostsze przejścia literowe i epsilon.

Wszystkie programy posiadają swoje wady i zalety. Z każdą iteracją zbliżaliśmy się do naszego celu, ale również odsłanialiśmy różne niedociągnięcia. Ostatecznie wszystkich algorytmów program 3 jest najbardziej odpowiedni, jeśli szukamy uniwersalnego rozwiązania na analizę wyrażeń regularnych.